Hierarchical Routed Sinkformer: Geometry-Shaped Representations for Compute-Adaptive Language Modeling

Micheal Bee

March 2026

Contents

Hierarchical Routed Sinkformer **3**
Geometry-Shaped Representations for Compute-Adaptive Language Modeling 3
Abstract 3

Part I: Core Architecture **4**

1. Motivation **4**

2. Design Principle **4**

3. Architecture Overview **5**
Core (Sections 4–8) 5
Extensions (Sections 9–10) 5
Training (Section 11) 5

4. Dual-Head Geometry Shaping **6**
Head A — Generative 6
Head B — Locality / task objective 6
Purpose: Representational Tension, Not Classification 6

5. Learned Routing **6**
Conservation Constraints 7
Formal Objective 7
Router Stability 8

6. Tiered Compute **9**
Tier 1 — Local mixing (default) 9
Tier 2 — Specialists 9
Tier 3 — Global attention (rare) 9
Empirical Support: The Two-Stage Hypothesis 9

7. Sink Memory Channel **10**
Salience Criteria 10

8. KV Cache as a Memory Hierarchy **11**
Core Architecture Summary . . . 11

Part II: Extensions **11**

9. Temporal Routing Cache (TRC) **11**
Simplification Note . . . 12

10. Recursive Thread Engrams (Long-Term Compression) **12**
Prior Empirical Results . . . 12
Engram Extraction as Energy Minimization . . . 13
When to Extract . . . 13
Engram Refinement . . . 13
The Memory Triangle . . . 14

Part III: Training **14**

11. Training Strategy **14**
The Problem: Five Gradients, One Backbone . . . 14
Principle: Phased Specialization with Differential Learning Rates . . . 14
Phase 1: Foundation (Backbone + Generative Head + Conv) . . . 15
Phase 2: Geometry Shaping (Add Locality Head + Router) . . . 15
Phase 3: Specialization (Experts + Sink) . . . 15
Phase 4: Compression (Engram Encoder) . . . 16
Phase 5: Engram Refinement *(Optional — see Section 13, Finding 3)* . . . 16
Self-Adaptive Loss Weighting . . . 17
Conservation Constraints on Routing . . . 17
Router Stability Diagnostics . . . 18
The Hopfield Justification . . . 18
Training Cost . . . 18
Summary . . . 18

12. Training Data **19**

Part IV: Evaluation **19**

13. Experimental Validation **19**
Setup . . . 19
Results . . . 20
v1 Ablation . . . 20
v2 Structural Prior Experiment . . . 21
v3 PEER + Routing Experiment . . . 21
v4 Unconditional PEER Experiment . . . 21
v5 Engram Replacement Experiment . . . 21
Analysis . . . 22
Finding 1: Engrams are the decisive component, but only with routing . . . 22
Finding 2: The router tax is real but not fatal . . . 22
Finding 3: Phased training dynamics and Phase 5 regression . . . 23
Finding 4: Causal convolution matters . . . 24

Finding 5: Routing is Nash equilibrium, not optimal transport 24
Finding 6: TRC provides modest routing stability 25
Finding 7: Fixed structural priors cannot substitute for learned routing 25
Finding 8: Expert capacity is not the bottleneck — but expert *role* matters 26
Finding 9: Unconditional PEER + reduced routing achieves the best result 26
Finding 10: Prepending engrams outperforms in-place replacement 27
Summary 28

14. Expected Benefits **29**
Validated (Section 13) 29
Projected (require larger-scale validation) 30

15. Minimal Model Hypothesis **30**

16. Ablation Order and Results **30**

17. Contributions **34**
Core Architecture 34
Extensions 34
Training 35
Empirical Findings 35

References (Author's Prior Work) **36**

Hierarchical Routed Sinkformer

Geometry-Shaped Representations for Compute-Adaptive Language Modeling

Abstract

Transformer language models process context as a flat token buffer and apply global attention uniformly. This design couples computation and memory: every token participates equally in every layer. As context grows, this causes quadratic compute, cross-topic interference, and degraded reasoning.

We argue that these failures arise from **missing routing and memory structure**, not insufficient parameter count.

We introduce the **Hierarchical Routed Sinkformer (HRS)**, a transformer architecture organized around a core principle: computation should be proportional to relevance. (The "Sink" refers to the sink memory channel, not Sinkhorn normalization — our empirical work demonstrated that Sinkhorn-based optimal transport is the wrong formalism for dual-objective routing; see Section 5.) The core architecture provides:

- dual-head geometry shaping to prevent representational collapse
- learned routing via Nash equilibrium between competing objectives
- tiered compute operators (convolution -> experts -> global attention)
- a sink channel for interference reduction

Two optional extensions address long-horizon memory:

- a Temporal Routing Cache (TRC) that amortizes routing across coherent spans

- compressed thread "engrams" for long-term memory with bounded growth

A phased training protocol with differential learning rates prevents gradient monopolization across HRS's heterogeneous components — addressing a failure mode we have empirically demonstrated in prior multi-component architectures.

This design decouples memory from compute: tokens are cheap to store but expensive to attend. By controlling attention participation rather than deleting history, HRS achieves better reasoning and lower compute. A four-axis ablation on WikiText-103 (14 configurations across v1–v4) demonstrates up to 65.2% perplexity reduction over a dense baseline (8.3 vs 23.83), with the best result achieved by separating sparse expert retrieval (PEER) as an unconditional FFN from a reduced 3-tier routing pathway — confirming that routing geometry and compression, not expert capacity, drive HRS performance. We hypothesize that a well-routed 1B HRS model can match or exceed dense 7B models at equivalent FLOPs on reasoning-heavy tasks through superior context curation.

Code: github.com/MikeyBeez/HRS

Part I: Core Architecture

---

1. Motivation

Current systems from organizations such as **Google** and **Anthropic** scale primarily by increasing parameters and context length.

However:

- storing tokens in KV cache is cheap (O(d))
- attending to all tokens is expensive (O(n.d))

Thus the bottleneck is not memory capacity but **indiscriminate attention**.

Flat attention leads to:

- context pollution across threads
- wasted FLOPs on irrelevant tokens
- degraded long-horizon reasoning

We propose that performance is limited more by **routing quality** than by parameter count.

---

2. Design Principle

Computation should be proportional to relevance.

Most tokens require only local processing. Few tokens require global reasoning.

Therefore, the model should:

1. cheaply process everything locally

2. escalate only important tokens to expensive compute
3. quarantine irrelevant information
4. compress old signal into summaries

This yields a hierarchical compute and memory system rather than a monolithic transformer.

The analogy is deliberate. In *The Context Window Is a Computer* (Bee 2026) and *The Protocol Harness: An Operating System Written in Protocols* (Bee 2025), we argued that the context window is not a document but RAM, and that without an operating system — mechanisms for managing what is in active memory, secondary storage, and the index — it is useless for complex tasks. HRS is that operating system realized as architecture:

- the router is a **scheduler**
- the sink is **swap space**
- the hot/warm/cold KV tiers are a **cache hierarchy**
- global attention is a **system call** — privileged and expensive
- engrams are **checkpoints**

This is not a metaphor applied after the fact. The architecture was designed from this framing.

---

3. Architecture Overview

HRS is organized as a **core** plus **extensions**.

Core (Sections 4–8)

1. Dual-Head backbone (geometry shaping)
2. Learned routing with conservation constraints
3. Tiered compute operators (conv -> experts -> attention)
4. Sink memory channel
5. KV cache as memory hierarchy

The core is self-contained. It delivers routing-based sparse attention, interference reduction, and tiered compute without requiring long-horizon memory.

Extensions (Sections 9–10)

6. Temporal Routing Cache (TRC)
7. Recursive thread engrams

These add long-context memory management. They require the core to be functioning but are independently valuable — either can be adopted without the other.

Training (Section 11)

A phased protocol that applies to both core and extensions.

---

4. Dual-Head Geometry Shaping

Head A — Generative

Standard next-token cross-entropy.

Head B — Locality / task objective

Predicts weak supervision signals:

- local window membership
- contrastive positives/negatives
- tool or code boundaries
- optional thread labels

Purpose: Representational Tension, Not Classification

Single-loss transformers exhibit embedding anisotropy and low effective rank. We have measured this directly: in *Generative Auxiliary Training as Anti-Collapse Regularization* (Bee 2026), single-objective fine-tuning collapsed effective rank by 98.7% — from 984 to 12.6 dimensions on AG News. Only 3 dimensions captured 95% of variance. The model compressed its representations down to barely enough to separate classes, destroying the distributional richness built during pre-training.

Adding the generative head restored effective rank to 1097, *exceeding* the pre-trained baseline. On AG News 4-class classification, accuracy improved from 90.76% to 95.54% — a 4.78 percentage point gain attributable entirely to representational preservation.

The mechanism is not additive but tensional. In *MCR2 as Language Model Regularizer* (Bee 2026), we tested expansion and compression objectives independently:

- expansion alone: validation perplexity 335.5 (worse than baseline 315.2)
- compression alone: 317.2 (worse than baseline)
- both together: 306.4 (2.8% improvement)

Neither objective helps alone. Each alone makes things worse. The improvement comes from the irreconcilable conflict: the generative head demands distributed representations for next-token prediction; the locality head demands clustered representations for routing. This tension prevents collapse while preserving the fine-grained structure that downstream components need.

---

5. Learned Routing

Each token/state receives a lightweight relevance score from a small router network.

The router decides:

- local processing
- specialist expert
- global attention
- sink

Similarity is learned (MLP/cosine), not fixed L2, avoiding brittle geometry assumptions.

Routing is soft and probabilistic rather than hard pruning.

Conservation Constraints

Routing tokens between tiers involves mixing operations. Unconstrained mixing amplifies signal magnitude exponentially with depth.

This is the same failure mode identified in DeepSeek's mHC architecture and analyzed in *Symmetry Constraints and Scaling Stability: A Noether-Inspired Framework* (Bee 2026) and *The Four-Lane Highway: Making Sense of DeepSeek's mHC Paper* (Bee 2026). At 27B parameters, unconstrained mixing amplified signals by up to 3000x, making training unstable.

Routing weights must satisfy conservation properties:

- all entries non-negative
- every row sums to 1 (each token's routing weights form a probability distribution)
- global tier utilization approximately uniform (no tier collapse)

Our initial design enforced full doubly stochastic constraints via the Sinkhorn-Knopp algorithm. This failed for two reasons — one practical, one theoretical.

The practical failure: Sinkhorn normalization on the actual routing matrix shape (B.T x n_tiers, where B.T » n_tiers) produces per-entry values of approximately 1/(B.T). With typical batch sizes (B=24, T=512), this yields routing weights of ~0.00008 — effectively zeroing out all tier outputs during evaluation and producing random routing during training where Gumbel noise dominates the near-zero logits.

The theoretical failure: Sinkhorn-Knopp solves entropy-regularized optimal transport — finding the minimum-cost assignment under a single objective. But HRS is a dual-objective system. The generative head (Head A) demands distributed representations for next-token prediction; the locality head (Head B) demands clustered representations for routing. These are competing objectives with no single cost function to minimize. The routing weights do not emerge from a transport plan — they emerge from a **Nash equilibrium** between the two heads. Optimal transport is the wrong formalism; game theory is the right one.

This distinction is not cosmetic. Optimal transport has a unique solution for a given cost matrix. A Nash equilibrium depends on the dynamics of the competing players — which is exactly why phased training (Section 11) matters. The order in which objectives are introduced determines which equilibrium the system converges to.

Formal Objective

The dual-head routing objective can be expressed as a constrained optimization with a dynamic Lagrange multiplier:

```
min_{P1, P2}  Cost(P1, P2)  +  lambda(theta) . D_KL(P1 || P2)
```

where P1 is the routing distribution induced by the generative head, P2 is the routing distribution induced by the locality head, theta are the learned model weights, and lambda(theta) is a learned scalar controlling the tension between the two objectives.

This formulation captures three essential properties:

1. **Two distributions, not one.** Optimal transport minimizes cost under a single distribution. Here, P1 and P2 are jointly optimized but irreducibly distinct — the generative head and locality head induce different preferences over routing, and the system must balance them.

2. **The tension is learned, not fixed.** lambda(theta) depends on the model's weights, making it adaptive. Early in training, when representations are unstable, the model may learn a small lambda (tolerating large divergence between the heads). As training progresses and the heads converge on a shared geometry, lambda can increase (penalizing divergence to stabilize the equilibrium). This connects directly to the self-adaptive loss weighting in Section 11, where sigmoid-activated scalar parameters serve as learned lambda values for each loss component.

3. **KL divergence measures productive disagreement.** D_KL(P1 || P2) quantifies how much the two heads disagree about routing. Some disagreement is essential — it is the source of the representational tension that prevents collapse (Section 4). Too much disagreement means the heads are fighting rather than composing. The learned lambda finds the productive balance.

In practice, this objective is realized through the combined loss function where each head's gradient competes through the shared backbone, and the routing weights emerge from the resulting equilibrium.

We implement the game-theoretic routing using three complementary mechanisms:

1. **Softmax normalization** ensures per-token routing weights are non-negative and sum to 1 (row constraint)
2. **Balance loss** (squared-mean formulation: n.Sigmap_i^2 where p_i is mean tier utilization) penalizes deviation from uniform global distribution (soft column constraint), acting as a regularizer on the equilibrium
3. **Gumbel-softmax** with temperature annealing provides differentiable discrete routing during training — the temperature schedule controls the sharpness of the equilibrium, analogous to annealing in game-theoretic learning

This formulation lets routing emerge from gradient competition between objectives rather than from solving a transport problem. A learnable per-dimension output gate (initialized at 0.1) further controls the magnitude of tier outputs relative to the residual stream, preventing any single tier from dominating the equilibrium.

Additionally, per-token **entropy regularization** prevents premature routing commitment. Without it, the router collapses to a fixed assignment within the first few thousand steps (routing entropy < 0.01), before the tiers have had time to specialize. With entropy regularization (weight 0.01), routing entropy decreases gradually from ~1.0 to ~0.1 over 40K steps — the system explores the equilibrium landscape before committing.

These conservation constraints are essential for scaling HRS beyond small models.

Router Stability

During training, the router will oscillate — assigning the same token to different tiers across consecutive steps. Not all oscillation is pathological.

Binary Node Oscillation in Neural Networks (Bee 2025) distinguished two types:

Type I (pathological): The router receives contradictory gradient signals. The same input is routed to experts in one batch and to sink in the next, not because the input is ambiguous but because the training signal is inconsistent. Symptom: high routing entropy that does not decrease with training.

Type II (meaningful): The input is genuinely ambiguous — a token that is relevant in some contexts and irrelevant in others. The router correctly oscillates because the optimal routing depends on context. Symptom: routing entropy decreases for most tokens but remains high for a learnable subset.

Diagnostic: Track per-token routing entropy across training. Healthy training shows global routing entropy decreasing, a long tail of high-entropy tokens for genuinely ambiguous inputs, and routing decisions for unambiguous tokens stabilizing within Phase 2 of training (see Section 11).

---

6. Tiered Compute

Instead of full attention everywhere, each layer provides multiple operators:

Tier 1 — Local mixing (default)

Depthwise convolution or small local attention.

- O(n) cost
- handles most tokens
- captures short-range structure

Tier 2 — Specialists

Parameter-efficient expert modules for domain skills.

Only activated when routed.

Tier 3 — Global attention (rare)

Full softmax attention for high-importance tokens.

Used sparingly.

This makes attention an **exception path**, not the default.

Empirical Support: The Two-Stage Hypothesis

This tiered design is supported by *Attention Builds Maps, Then Reads Them: The Two Stages of Transformer Intelligence* (Bee 2025), which demonstrated that hybrid models — replacing later-layer attention with static functions — lost only 1.4% quality while gaining 58% speed. Early layers build geometric structure via attention; later layers navigate established geometry and can use cheaper operators.

Further evidence comes from *Static Functions Can Approximate Deep Attention Layers* (Bee 2025), where per-head MLP approximations explained up to 92.6% of variance in deeper-layer attention

head outputs. Deeper layers compute increasingly predictable functions — they do not need the full dynamic attention mechanism.

Together, these findings justify the tier structure: global attention in early layers where geometry is forming, cheap operators in later layers where geometry has stabilized.

---

7. Sink Memory Channel

Transformers lack a forgetting mechanism; irrelevant tokens continue to participate in attention and interfere with reasoning.

We introduce a sink pathway:

Low-relevance tokens are routed toward:

- sink tokens
- or a sink subspace
- or low-priority KV tier

They remain stored but minimally attended.

This:

- prevents interference
- avoids brittle deletion
- allows recovery if later needed

Memory is preserved while compute is reduced.

Salience Criteria

What determines which tokens are sunk? *Memory, Salience, and Why Your LLM Forgets* (Bee 2025) proposed measurable dimensions of token persistence:

- **Information density:** High-density tokens (numbers, rare words, IDs) recalled at ~89%; low-uniqueness tokens at ~34%
- **Position:** U-shaped recall curve — early and late tokens persist, middle tokens are most vulnerable
- **Structure:** Tokens wrapped in delimiters or markers show higher persistence
- **Geometry:** Tokens with high projection magnitude in the residual stream survive compression better

The sink router should learn these dimensions implicitly through the dual-head training signal. The locality head's contrastive objective naturally creates separation between high-salience and low-salience tokens — the sink channel exploits this learned separation rather than requiring hand-engineered rules.

---

8. KV Cache as a Memory Hierarchy

Maintaining KV cache is cheap compared to attention.

Therefore we keep all tokens but control **participation** rather than **existence**.

We maintain tiers:

- hot: active thread
- warm: specialists
- cold: sink

All live in KV; only routing controls which are attended.

This decouples memory size from compute cost.

Current approaches treat compression as reactive — summarize after overflow. The correct approach is proactive hierarchical memory with tiered access. HRS's learned routing serves as a memory controller, deciding what lives in active working memory (hot), what is available for specialist retrieval (warm), and what is stored but not attended (cold).

This reframes the scaling problem: we do not need longer context windows, we need **better context management**.

Core Architecture Summary

The core HRS — dual-head geometry, router, tiered compute, sink, KV hierarchy — is a complete system. It delivers:

- geometry-aware sparse attention
- interference reduction via the sink
- mostly O(n) compute with rare O(n^2) escalation
- all tokens preserved, participation controlled

What it does not address is **long-horizon memory**. Conversations still grow linearly. The extensions in Part II solve this.

Part II: Extensions

---

9. Temporal Routing Cache (TRC)

Routing decisions exhibit strong temporal correlation (e.g., code blocks, math derivations).

We cache routing indices across spans and reuse them for multiple tokens.

Routing is recomputed only on distribution shifts.

This amortizes routing overhead and reduces compute significantly in coherent segments.

Simplification Note

A simpler alternative — low-pass filtering router logits across adjacent tokens — may capture most of the TRC's benefit without explicit caching or statefulness. We recommend benchmarking low-pass filtering as a baseline before adopting the full TRC. If the gap is marginal, the simpler approach is preferable. TRC adds statefulness across tokens, routing inertia, and harder credit assignment; these costs are justified only by proportional gains.

---

10. Recursive Thread Engrams (Long-Term Compression)

Even with routing and sinking, long conversations grow linearly.

We introduce **thread engrams**:

Periodically, the locality head extracts a compressed summary of the active thread:

1. pool hidden states from the model's middle layers
2. compress via small encoder
3. produce fixed-size engram vector(s)
4. append back to context as memory tokens

Older raw tokens can then be down-tiered to sink.

This provides:

- bounded memory growth
- persistent long-term knowledge
- cheaper reasoning over history

The model learns to treat engrams as high-signal summaries.

Prior Empirical Results

Engrams are not speculative. In *Engrams: Learned Semantic Compression for Transformers* (Bee 2026), we demonstrated that middle-layer hidden state extraction, compressed to 32 engram vectors from 8,192 source tokens (256x compression), achieved 96% factual accuracy versus 80% for full-text RAG retrieval with 64x fewer tokens.

The critical design choice is extraction layer. A layer sweep confirmed that the middle third of the model is optimal — deep enough to encode semantic relationships, not so deep that representations have specialized for next-token prediction. In a 7B model, layer 16 (of 32) produced the best engrams.

Three mechanisms explain why compressed engrams outperform full-text retrieval:

- **Signal concentration:** Mean-pooling blends information from multiple tokens, increasing density
- **Noise reduction:** Compression strips formatting, tangential text, and syntactic scaffolding
- **Representation alignment:** Injecting at the embedding layer in the model's native representational format avoids the lossy text->token->embedding round-trip that RAG requires

Further investigation in *Engrams Don't Inject Information — They Retrieve It* (Bee 2026) showed that engrams function as retrieval cues for information already stored in the model's weights, not as information containers. *What We Got Wrong About Engram Steering* (Bee 2026) and *Golden Ratio Engram: A Corrected Account* (Bee 2026) documented iterative corrections to the extraction and injection procedure — the kind of empirical refinement that distinguishes a tested technique from a theoretical proposal.

Engram Extraction as Energy Minimization

The engram is not ad-hoc compression. *Recursive Refinement as Approximate Hopfield Dynamics* (Bee 2026) showed that a single attention pass is mathematically equivalent to a Hopfield update rule, and that recursive passes converge toward stored attractor patterns. Small recursive models achieved 99.2% improvement over single-pass baselines on word-level language modeling.

Critically, the bottleneck compression in engrams filters noise *geometrically* — by excluding low-variance dimensions — rather than by attenuation. This is strictly more effective than temperature scaling. The engram is a compressed representation of an attractor basin's neighborhood in activation space: the essential structure of a thread, with the noise excluded by geometry.

When to Extract

Engram extraction should not follow a fixed schedule. *Differentiable Time: When Neural Networks Learn They Are Finished* (Bee 2026) proposed using Jacobian sensitivity via Hutchinson's stochastic trace estimator to detect when layer computations have converged. When a thread's representations stabilize — sensitivity drops below a learned threshold — the representations are ready for compression.

This gives the model a self-assessed signal for when a thread has been "understood" well enough to compress. The same mechanism serves double duty: it triggers engram extraction at inference time and provides the Phase 3->4 transition criterion during training (Section 11).

Engram Refinement

Standard training will underutilize engrams. *Iterative Refinement: Breaking Through Convergence Plateaus* (Bee 2025) demonstrated that freezing converged representations and retraining the classifier head improved validation loss by 24-28% — with no additional parameters or data. The mechanism: jointly trained classifiers reach a local optimum that leaves extractable information on the table.

The same principle applies to engram consumers. After the full system converges:

1. Freeze the engram encoder
2. Reinitialize the connection weights from engram inputs to downstream layers
3. Retrain those downstream layers

The retrained layers will extract more information from the same compressed representations. If the 24-28% improvement transfers to engram consumption — and the Hopfield dynamics analysis suggests it should, since the engram representations form the same kind of stable attractors that iterative refinement exploits — skipping this step wastes a quarter of the engram's informational capacity.

The Memory Triangle

Routing handles **selection**. The sink handles **forgetting**. Engrams handle **compression**.

That is the full memory triangle. Each addresses a distinct failure mode of flat attention:

- without selection, compute is wasted on irrelevant tokens
- without forgetting, irrelevant tokens interfere with reasoning
- without compression, history grows without bound

The core (Part I) delivers the first two. Engrams complete the third.

Part III: Training

---

11. Training Strategy

The Problem: Five Gradients, One Backbone

HRS combines five component types in a shared representational space. Each produces its own loss gradient. All gradients flow through the same backbone. Naive end-to-end training will fail.

This is not speculation. In *We Stacked 3 AI Upgrades. The Combined System Was Worse Than Using Just 1* (Bee 2026), we demonstrated that combining three complementary architectural improvements produced worse performance than any single improvement alone. The failure mode was gradient monopolization: the fastest-learning component captured representational real estate before slower components could specialize, and subsequent components optimized around an already-distorted geometry.

HRS is more complex than that three-component system. Without a principled training strategy, the same failure mode will occur at larger scale.

Principle: Phased Specialization with Differential Learning Rates

We adopt the phased training approach from *Phased Specialization: Unlocking Hybrid Sequence Models* (Bee 2026), adapted for HRS's component architecture.

Heterogeneous components learn at different natural speeds. Convolutions converge fast. Expert routing converges slowly. Attention is somewhere between. If all components train at the same learning rate, the fast learners monopolize the gradient signal before the slow learners have established useful representations.

The fix is not sequential freezing (which creates brittle handoff points) but **differential learning rates with soft phases**. No component is ever fully frozen (except the engram encoder in early phases). Minimal learning rates (1e-5 to 1e-6) maintain compatibility with the evolving backbone without allowing distortion. This was the key finding in the stacking experiments: fully frozen components fall out of sync and cause instability when later unfrozen.

Phase 1: Foundation (Backbone + Generative Head + Conv)

Goal: Establish stable representations and local structure.

Component	Learning Rate	Status
Backbone	full LR	lead
Generative head (Head A)	full LR	lead
Conv operators	full LR	lead
Locality head (Head B)	0.1x LR	warming
Router	0.01x LR	minimal
Expert modules	0.01x LR	minimal
Sink channel	0.01x LR	minimal
Engram encoder	frozen	off

The backbone learns basic language structure. Convolutions establish local mixing patterns. The generative head provides the primary training signal.

Duration: Until backbone validation loss plateaus.

Phase 2: Geometry Shaping (Add Locality Head + Router)

Goal: Shape representational geometry for routing via dual-head tension.

Component	Learning Rate	Status
Backbone	0.5x LR	supporting
Generative head (Head A)	full LR	lead
Conv operators	0.5x LR	supporting
Locality head (Head B)	full LR	**lead**
Router	0.5x LR	**ramping**
Expert modules	0.1x LR	warming
Sink channel	0.1x LR	warming
Engram encoder	frozen	off

The locality head comes online as co-lead. Its weak supervision signal shapes the backbone geometry to form the clusters that routing will exploit. The router begins learning at 0.5x LR, exploiting the emerging cluster structure. It does not need the geometry to be perfect — it needs it to be forming.

Transition criterion: Monitor effective rank via SVD on backbone hidden states. When effective rank stabilizes at or above the Phase 1 baseline, geometry shaping is working. If effective rank drops below 50% of baseline, reduce locality head LR — the auxiliary objective is too aggressive.

Phase 3: Specialization (Experts + Sink)

Goal: Activate specialist computation and forgetting.

Component	Learning Rate	Status
Backbone	0.3x LR	supporting
Generative head (Head A)	full LR	lead
Locality head (Head B)	0.5x LR	supporting
Conv operators	0.3x LR	supporting
Router	full LR	**lead**
Expert modules	full LR	**lead**
Sink channel	full LR	**lead**
Engram encoder	frozen	off

The router, experts, and sink come online together. The router has had two phases of warmup and a forming cluster geometry to work with.

Diagnostic: Monitor output magnitudes per component. If one component's output norm exceeds 3x the others', it is monopolizing. Reduce its LR by 50%. This was the primary failure signal in the stacking experiments.

Note: For core-only HRS (without extensions), training ends here. The core delivers geometry-aware sparse attention, tiered compute, and interference reduction. Phases 4 and 5 apply only when engrams are included.

Phase 4: Compression (Engram Encoder)

Goal: Learn long-term memory compression. *Extension phase — requires engram module.*

Component	Learning Rate	Status
Backbone	0.1x LR	supporting
Generative head (Head A)	0.5x LR	supporting
Locality head (Head B)	0.3x LR	supporting
Conv operators	0.1x LR	supporting
Router	0.5x LR	supporting
Expert modules	0.5x LR	supporting
Sink channel	0.5x LR	supporting
Engram encoder	full LR	**lead**

The engram encoder trains last because it depends on all other components being stable. Training engrams on unstable representations produces unstable compressions.

Transition criterion from Phase 3: Jacobian sensitivity (Hutchinson's estimator) across routing and expert layers drops below learned threshold — the same convergence signal described in Section 10.

Phase 5: Engram Refinement *(Optional — see Section 13, Finding 3)*

Goal: Maximize information extraction from compressed representations. *Extension phase.*

Freeze the engram encoder. Reinitialize downstream connection weights. Retrain the layers that consume engrams. (See Section 10, Engram Refinement.)

Duration: Fixed — equivalent to Phase 4 duration.

Empirical note: Phase 5 consistently degrades performance at the 50M-parameter scale tested (Section 13). All three engram variants show best perplexity at the end of Phase 4, with Phase 5 adding +0.62 to +1.40 PPL regression. We recommend treating this phase as optional pending validation at larger scale.

Self-Adaptive Loss Weighting

Manual tuning of loss weights across four or more objectives is fragile. We adopt the approach from *Self-Adaptive Loss Weighting Through Network Node Designation* (Bee 2025): designate specific network nodes whose sigmoid-activated outputs serve as learnable loss scaling factors.

For each loss component L_i:

```
w_i = sigmoid(node_i) + epsilon
L_total = sum(w_i * L_i)
```

where node_i is a dedicated scalar parameter updated by backpropagation, and epsilon (1e-4) prevents any objective from being fully suppressed.

Phased differential LRs provide the coarse schedule. Adaptive loss weights provide fine-grained balancing within each phase. The two mechanisms are complementary.

Conservation Constraints on Routing

Routing tokens between tiers involves mixing operations. Unconstrained mixing amplifies signal magnitude exponentially with depth (see Section 5 for the DeepSeek mHC precedent).

Routing conservation is enforced through three complementary mechanisms:

1. **Per-token softmax** ensures routing weights are non-negative and sum to 1
2. **Balance loss** (n.Sigmap_i^2) penalizes global tier imbalance, preventing routing collapse
3. **Entropy regularization** (negative mean per-token entropy) prevents premature commitment

During training, Gumbel-softmax with annealed temperature (tau: 1.0 -> 0.3 over 40K steps) provides differentiable discrete routing. At evaluation, deterministic softmax with minimum temperature produces stable assignments.

A learnable per-dimension **tier output gate** (initialized at 0.1) controls the magnitude of tier outputs relative to the residual stream. Without this gate, tier outputs (magnitude ~1.0) dominate the residual addition (baseline MLP outputs ~0.007 after GPT-2-style output scaling), causing effective rank collapse. The gate learns to scale tier contributions to match the residual stream's magnitude profile.

Empirical note: Our initial implementation used Sinkhorn-Knopp normalization for full doubly stochastic constraints. This failed both practically (non-square matrix pathology) and theoretically — Sinkhorn solves optimal transport under a single cost function, but the dual-head system is a two-player game whose routing equilibrium emerges from competing gradients, not from a transport plan. See Section 5 for the full analysis and Section 13 for empirical validation.

Router Stability Diagnostics

During training, the router will oscillate — assigning the same token to different tiers across consecutive steps. Not all oscillation is pathological.

Binary Node Oscillation in Neural Networks (Bee 2025) distinguished two types:

Type I (pathological): The router receives contradictory gradient signals. The same input is routed to experts in one batch and to sink in the next, not because the input is ambiguous but because the training signal is inconsistent. Symptom: high routing entropy that does not decrease with training.

Type II (meaningful): The input is genuinely ambiguous — a token that is relevant in some contexts and irrelevant in others. The router correctly oscillates because the optimal routing depends on context. Symptom: routing entropy decreases for most tokens but remains high for a learnable subset.

Diagnostic: Track per-token routing entropy across training. Healthy training shows:

- global routing entropy decreasing
- a long tail of high-entropy tokens (genuinely ambiguous inputs)
- routing decisions for unambiguous tokens stabilizing within Phase 2

If global routing entropy fails to decrease by Phase 3, the router is receiving contradictory gradients. Reduce the locality head LR — the geometry-shaping signal is confusing the router rather than helping it.

The Hopfield Justification

The phased strategy has a theoretical basis beyond empirical convenience. Each phase corresponds to establishing a new level of attractor structure in the model's energy landscape:

- Phase 1 creates the base energy surface (language structure)
- Phase 2 sculpts basins (semantic clusters via dual-head tension)
- Phase 3 creates routing channels between basins (expert specialization)
- Phase 4 creates compressed summaries of basin neighborhoods (engrams)
- Phase 5 refines the readout of those summaries

Each phase must wait for the previous landscape to stabilize. Training all phases simultaneously is equivalent to sculpting valleys in a landscape that is still forming — the valleys shift before they can be used.

Training Cost

Total training cost is approximately 1.4-1.6x a standard single-phase run of equivalent total steps (1.2x for core-only without Phases 4-5). The overhead comes from reduced LR utilization in early phases. The alternative — naive end-to-end training — produces the stacking failure: components that fight rather than compose.

Summary

Phase	Duration	Lead Components	Key Metric
1: Foundation	Until val loss plateaus	Backbone, GenHead, Conv	Validation loss
2: Geometry	Until effective rank stabilizes	LocalityHead, Router (ramping)	Effective rank (SVD)
3: Specialization	Until routing entropy stabilizes	Router, Experts, Sink	Routing entropy, output magnitudes
4: Compression*	Until engram reconstruction converges	Engram encoder	Reconstruction quality
5: Refinement*	Fixed (= Phase 4 duration)	Downstream engram consumers	Val loss with engram context

*Phases 4-5 apply only when engram extension is included.

---

12. Training Data

Precise thread labels are unnecessary.

Weak supervision for Head B suffices:

- local window positives
- distant negatives
- structural cues (tools/code)

This allows training without proprietary chat logs and avoids brittle labeling.

Part IV: Evaluation

---

13. Experimental Validation

Setup

We validated HRS on WikiText-103 (118M tokens) using a 4-layer, 8-head, d_model=512 transformer (38M parameters for the dense baseline, up to 50M with all HRS components). All configurations were trained for 50,000 steps with effective batch size 48, learning rate 3e-4 with cosine decay, on a single NVIDIA RTX 5070 Ti (16GB VRAM). BF16 mixed precision was used throughout.

v1 ablation (8 configs). Seven ablation configurations were trained, each adding one component. Two additional configurations were run after correcting an implementation error in the original config 7 (see note below):

1. **dense_baseline** — Standard transformer, cross-entropy only (38M params)

2. **dual_head** — + locality head with InfoNCE contrastive loss
3. **dual_head_router** — + learned router with tiered compute (conv/expert/attn/sink)
4. **dual_head_router_sink** — + sink channel enabled
5. **full_core** — + phased training protocol (Phases 1-3)
6. **full_hrs** — + engram encoder (Phases 1-5, 50M params)
7. **full_hrs_refined** — + engram refinement (freeze encoder, reinitialize injector at P5)
8. **full_hrs_trc** — full_hrs + Temporal Routing Cache (causal moving average, window=8)

v2 experiment (4 configs). To test whether routing could be replaced by a fixed structural prior, we built a second architecture: attention layers 1-2, convolution layers 3-4 (no router), with Parameter Efficient Expert Retrieval (PEER) replacing the standard MLP. PEER uses product-key retrieval over 262,144 single-neuron experts (512 sub-keys x 2), activating 128 per token (8 heads x 16 top-k), achieving 99.95% sparsity.

9. **v2_attn_conv** — Attention->Conv backbone, standard MLP (36.8M params)
10. **v2_attn_conv_dual** — + dual-head (locality)
11. **v2_attn_conv_peer** — + PEER FFN replacing MLP (167M params, 138M sparse)
12. **v2_full** — + engrams + phased training (169.7M params)

v3 experiment (1 config). To test whether PEER's sparse experts improve upon v1's 4-expert MoE within the proven routing framework, we performed a single surgical substitution: PEER replaces ExpertTier in the v1 HRSBlock. All other components — router, conv tier, attention tier, sink tier, dual-head, engrams, 5-phase training — remain identical to full_hrs_refined. Batch size was reduced from 24 to 8 (with gradient accumulation increased to 6) to fit the larger model in 16GB VRAM.

13. **v3_full** — v1 routing framework + PEER as expert tier + engrams (175.6M params)

v4 experiment (1 config). The v3 result revealed that PEER and attention compete for routing weight — they serve different roles (per-token feature transformation vs. token mixing) but are forced into the same routing decision. v4 separates them: PEER becomes an unconditional FFN (runs for every token in every layer, replacing the standard MLP entirely), while the router drops to 3 tiers (conv, attention, sink) and only decides the attention pathway. Every token receives both its routed pathway output and unconditional PEER output, summed. All other components remain identical to v3.

14. **v4_full** — 3-tier routing (conv/attn/sink) + unconditional PEER FFN + engrams (175.6M params)

Results

v1 Ablation

#	Configuration	Val PPL	Best PPL (Phase)	Delta vs Baseline	Eff. Rank	Routing Entropy	Tier Dist. (c/e/a/s)
1	dense_baseline	**23.83**	—	—	391.5	—	—
2	dual_head	**24.08**	—	+1.0%	90.1	—	—
3	dual_head_router	**31.63**	—	+32.7%	13.3	0.026	21 / 29 / 32 / 18
4	dual_head_router_sink	**31.90**	—	+33.9%	12.3	0.035	20 / 32 / 29 / 19

#	Configuration	Val PPL	Best PPL (Phase)	Delta vs Baseline	Eff. Rank	Routing Entropy	Tier Dist. (c/e/a/s)
5	full_core	**37.84**	—	+58.8%	19.1	0.055	24 / 31 / 25 / 20
6	full_hrs	**12.04**	10.64 (P4)	-49.5%	43.1	0.101	26 / 30 / 32 / 12
7	full_hrs_refined	**10.51**	**9.19** (P4)	**-55.9%**	—	—	—
8	full_hrs_trc	**10.05**	**9.43** (P4)	**-57.8%**	—	—	—

Note on config 7: The original ablation run labeled "full_hrs_refined" contained an implementation error — no actual engram refinement was performed (the Phase 5 freeze/reinitialize protocol was not triggered). That run (val_ppl 10.04) was effectively a second full_hrs run, confirming substantial run-to-run variance across engram configurations (range: 10.0–12.0). The results above reflect the corrected implementation where the engram encoder is frozen and the injector type embedding is reinitialized at the Phase 5 transition.

Best PPL column: For engram configurations (6-8), the best validation perplexity occurs at the end of Phase 4, before Phase 5 degrades performance. This is discussed in Finding 3 below.

v2 Structural Prior Experiment

#	Configuration	Params	Val PPL	Delta vs Baseline
9	v2_attn_conv	36.8M	**25.7**	+7.8%
10	v2_attn_conv_dual	36.8M	**26.2**	+9.9%
11	v2_attn_conv_peer	167M	**24.1**	+1.1%
12	v2_full	169.7M	**27.2**	+14.1%

v3 PEER + Routing Experiment

#	Configuration	Params	Val PPL	Best PPL (Phase)	Delta vs Baseline	Tier Dist. (c/p/a/s)
13	v3_full	175.6M	**14.33**	**9.46** (P4)	**-60.3%**	26 / 36 / 31 / 7

v4 Unconditional PEER Experiment

#	Configuration	Params	Val PPL	Best PPL (Phase)	Delta vs Baseline	Tier Dist. (c/a/s)
14	v4_full	175.6M	**9.6**	**8.3** (P4)	**-65.2%**	30 / 43 / 26

v5 Engram Replacement Experiment

#	Configuration	Params	Val PPL	Best PPL (Phase)	Delta vs Baseline	Tier Dist. (c/a/s)
15	v5_replace	176.2M	**10.4**	**10.4** (P4)	**-56.3%**	30 / 52 / 18

v5 tests whether *replacing* stale context windows with upsampled engrams (in-place blend) outperforms *prepending* engrams (extra KV entries). A learnable gate sigma(alpha.(loss-theta)) uses EMA-cached per-token loss from the previous step to decide which 128-token windows to replace. The gate threshold theta and sharpness alpha are learnable parameters, frozen during P1–P3 and unfrozen in P4. Phase 5 is skipped (38K total steps). The architecture is otherwise identical to v4.

Analysis

Finding 1: Engrams are the decisive component, but only with routing

The clearest result is the discontinuity between configurations 5 and 6. Adding routing without engrams (configs 3-5) consistently *increases* perplexity by 33-59% — the "router tax" exceeds the routing benefit. Adding engrams (config 6) produces a 49.5% *decrease* — from 23.83 to 12.04 at end of training (and to 10.64 at best). The best-of-run result across all engram configurations reaches 8.3 (config 14, v4_full at end of Phase 4), a 65.2% reduction from baseline. This is not an incremental gain from one more component. The engram system transforms what the architecture can do.

The mechanism is visible in effective rank. Routing alone collapses effective rank from 391.5 (baseline) to 12-19 — the router, even when healthy, concentrates representations into a low-dimensional routing-optimal subspace. Engrams restore effective rank to 43-44, recovering representational diversity through the compression bottleneck. The engram's lossy compression acts as a geometric regularizer, preserving the distributional structure that routing alone discards.

The v2 experiment confirms the converse: engrams without routing fail. When engrams are added to a fixed structural prior (v2_full, config 12), perplexity worsens from 24.1 to 27.2 — a 14.1% increase over baseline. Without the router creating structured geometry, the engram encoder has no organized representation to compress. The relationship is not "engrams help" but "routing creates geometry that engrams can compress." Remove either half and the system fails.

Finding 2: The router tax is real but not fatal

Configurations 3-5 demonstrate that routing machinery — the router MLP, tier operators, balance/entropy losses — imposes meaningful overhead at this model scale. The 38M-parameter model does not have enough capacity for the router to find savings that offset the additional parameters and optimization complexity.

However, all routed configurations show healthy routing dynamics: all four tiers remain active throughout training (no tier collapse), routing entropy decreases smoothly from ~1.0 to ~0.03, and the tier distribution stabilizes by step 10K. The routing machinery works correctly — it simply does not help perplexity without the engram system to exploit the structured representations it creates.

This suggests the router's value is infrastructural: it creates the organized representational geometry that engrams need to compress effectively. Routing without engrams is scaffolding without a building.

The v2 experiment provides the complementary test. Replacing routing with a fixed structural prior (attention layers 1-2, convolution layers 3-4) yields baseline-equivalent perplexity even with PEER's 262K sparse experts (24.1 vs 23.83). The structural prior is a reasonable architectural choice — it matches known patterns about early vs. late layer functions — but it does not create the dynamic, token-dependent geometry that learned routing produces. The result: engrams have nothing structured to compress, and adding them hurts (27.2 vs 24.1).

Finding 3: Phased training dynamics and Phase 5 regression

The `full_hrs` trajectory reveals how the phased protocol shapes training:

Phase	Steps	Best Val PPL	Tier Distribution	Observation
P1 (Foundation)	1-8K	27.0	68/10/20/3	Conv-dominated bootstrap
P2 (Geometry)	8-16K	15.1	36/28/25/11	Tiers rebalance, PPL drops sharply
P3 (Specialization)	16-26K	14.8	27/28/31/14	Modest improvement, routing stabilizes
P4 (Compression)	26-38K	11.1	27/30/31/12	Engrams activate — PPL drops 25%
P5 (Refinement)	38-50K	12.0	26/30/32/12	Regression: +0.9 PPL

Phase 1 reveals an unexpected pattern: the router initially sends 68% of tokens to the conv tier. This is rational — during foundation training, the conv tier is the only tier with meaningful gradients (experts and attention start with near-random weights). The router learns to route around untrained components.

Phase 4 produces the largest single-phase improvement (14.8 -> 11.1, a 25% drop). This is the engram activation phase, confirming that the compressed memory system is the primary driver of HRS performance.

Phase 5 consistently degrades performance across all engram configurations. This pattern holds for all five variants tested (v5 skips P5 entirely based on this evidence):

Configuration	Params	Best PPL (end P4)	Final PPL (end P5)	P5 Regression
full_hrs (v1)	50M	10.64 @ 38K	12.04	+1.40
full_hrs_refined (v1)	50M	9.19 @ 33K	10.51	+1.32
full_hrs_trc (v1)	50M	9.43 @ 36K	10.05	+0.62
v3_full	175.6M	9.46 @ 29K	14.33	+4.87
v4_full	175.6M	8.3 @ 31K	9.6	+1.30
v5_replace	176.2M	10.4 @ 37K	10.4 (no P5)	**0.00**

The refinement protocol — freezing the engram encoder and retraining downstream consumers — does not help: `full_hrs_refined` regresses by +1.32, comparable to the unrefined `full_hrs` (+1.40). TRC shows the smallest regression (+0.62), suggesting temporal routing smoothing provides

some stability during the phase transition. The v3 model shows the *largest* regression (+4.87), suggesting that Phase 5 disruption scales with coupled parameter count when PEER competes for routing weight.

Notably, v4 regresses by only +1.30 despite having the same 175.6M parameters as v3 (+4.87). The difference: in v4, PEER runs unconditionally and does not participate in routing, so freezing the engram encoder does not disrupt the PEER-routing coupling that exists in v3. This supports hypothesis (b) below — disruption is proportional to the degree of coupling between the frozen component and the rest of the system, not simply to total parameter count.

This is a clear negative result for Phase 5 at this model scale. Possible explanations: (a) the 24-28% iterative refinement improvement observed in prior work requires larger models or longer training to manifest; (b) freezing the encoder disrupts the joint optimization dynamic — the engram encoder, router, and backbone form a coupled system, and freezing one part shifts the equilibrium in ways that require more than 12K steps to recover; (c) the Phase 5 learning rate schedule (backbone at 0.3x, heads at 0.5x) is too aggressive for stable refinement. The v3-vs-v4 comparison strengthens hypothesis (b): with identical parameter counts, v3 regresses 3.7x more than v4 because v3's PEER is coupled through routing while v4's PEER is independent. We recommend that future work either eliminate Phase 5 or investigate gentler transitions (e.g., gradual encoder LR decay rather than hard freeze).

Finding 4: Causal convolution matters

During development, we discovered that symmetric convolution padding (seeing 3 future tokens with kernel_size=7) allowed the router to route 99%+ of tokens to the conv tier, achieving val_ppl of 1.1 — the model was cheating by reading the future. After correcting to causal (left-only) padding, conv tier usage dropped to a healthy 20-25%. This confirms that the router is sensitive to tier capabilities and will exploit any information advantage. Causal padding is essential for autoregressive HRS.

Finding 5: Routing is Nash equilibrium, not optimal transport

The most theoretically significant finding was the Sinkhorn failure. Our initial implementation used Sinkhorn-Knopp normalization to enforce doubly stochastic routing constraints, following the standard optimal transport formalism used in MoE routing literature. This failed catastrophically — but not only for the practical reason (non-square matrix pathology). The deeper issue is that **Sinkhorn solves the wrong problem**.

Sinkhorn-Knopp finds the entropy-regularized optimal transport plan — the minimum-cost assignment under a single objective. But HRS is a dual-objective system. The generative head pulls representations toward distributed geometry; the locality head pulls toward clustered geometry. The router's assignment emerges from the equilibrium of these competing gradients, not from minimizing any single transport cost. This is a Nash equilibrium, not an optimal transport solution.

The distinction has practical consequences:

1. **No unique optimum.** Optimal transport has a unique solution for a given cost matrix. A Nash equilibrium depends on the dynamics of competition — which is why phased training matters (the order of objective introduction determines which equilibrium the system finds).

2. **Sinkhorn normalization destroys the game.** On routing matrices of shape (B.T, n_tiers) where B.T=12,288 and n_tiers=4, Sinkhorn produces per-entry values of ~0.00008. But

even with correct dimensions, Sinkhorn would impose a single-objective constraint on a multi-objective system, suppressing the competitive dynamics that produce useful routing.

3. **Game-theoretic formulation works.** Softmax (per-player normalization) + balance loss (equilibrium regularizer) + entropy regularization (exploration) + Gumbel-softmax (annealed commitment) — these are game-theoretic mechanisms, not transport mechanisms. They let the equilibrium emerge rather than computing it.

Two additional implementation details proved critical:

- **Tier output scaling.** Without a learnable output gate, tier outputs (~1.0 magnitude) overwhelm the residual stream (baseline MLP outputs ~0.007 after GPT-2 scaling). A per-dimension gate initialized at 0.1 allows the model to learn appropriate scaling. Without it, effective rank collapses from 64 to 11 within 5K steps.
- **Entropy regularization prevents premature commitment.** Without the entropy loss term (weight 0.01), routing entropy drops below 0.01 within 3K steps, freezing the router into a fixed assignment before tiers have specialized. With entropy regularization, entropy decreases gradually (1.0 -> 0.1 over 40K steps), allowing the router to explore the equilibrium landscape throughout training.

Finding 6: TRC provides modest routing stability

The Temporal Routing Cache (config 8) applies a causal moving average over router logits with window size 8, smoothing routing decisions across adjacent tokens. Compared to the identical architecture without TRC (config 6), the TRC variant achieves a better best-of-run PPL (9.43 vs 10.64) and a smaller Phase 5 regression (+0.62 vs +1.40).

However, cross-run variance is substantial: the three full_hrs variants span 10.05–12.04 at end of training and 9.19–10.64 at best. The TRC improvement is within this variance band. A definitive assessment requires multiple seeds per configuration. What TRC clearly demonstrates is that temporal smoothing does not *hurt* — the low-pass filter imposes a mild inductive bias (adjacent tokens should route similarly) that is compatible with language's temporal coherence structure. At longer sequence lengths where routing volatility may increase, TRC's stabilizing effect is likely more pronounced.

Finding 7: Fixed structural priors cannot substitute for learned routing

The v2 experiment (configs 9-12) tested whether a reasonable fixed architecture — attention in early layers, convolution in later layers — could replace learned routing. This structural prior is well-motivated: prior work shows early attention heads learn position-dependent patterns while deeper heads compute near-static functions.

The results are unambiguous: no v2 configuration matches the v1 dense baseline. Without PEER, the structural backbone slightly underperforms (25.7 vs 23.83). PEER's sparse retrieval nearly recovers baseline performance (24.1), but adding engrams makes things *worse* (27.2) — the opposite of the v1 pattern where engrams are transformative. The critical difference is that learned routing creates *token-dependent* geometry: different tokens are routed to different compute pathways based on content, creating the structured representations that engrams can compress. A fixed structural prior applies the same pathway to all tokens regardless of content, producing representations with no exploitable structure for compression.

Finding 8: Expert capacity is not the bottleneck — but expert *role* matters

The v3 experiment (config 13) performed a controlled test: replace v1's 4-expert MoE (1.05M parameters per layer) with PEER's 262K sparse experts (138M parameters total) while keeping everything else identical — same router, same tiers, same dual-head, same engrams, same 5-phase training schedule.

Result: v3 achieves best val_ppl 9.46 (step 29K, early P4) versus v1's 9.19 (step 33K, late P4). With 3.5x more total parameters and 130x more addressable experts, PEER produces essentially the same perplexity. The router learned to favor PEER — allocating 36% of routing weight to the expert tier at peak (vs ~30% for v1's small MoE) — but the additional expert capacity did not translate to lower perplexity.

This initially appeared to be a pure negative result — expert capacity doesn't matter. But the v4 experiment (config 14) reveals a subtlety: **expert *role* matters even when expert *capacity* doesn't.** v4 uses the same PEER with the same 262K experts, but moves it out of routing competition into an unconditional FFN role. Result: 8.3 best PPL — a 9.7% improvement over v1 and 12.2% over v3, with identical PEER parameters.

The explanation: in v3, PEER competes with attention for routing weight. Both perform valuable but different functions — PEER handles per-token feature transformation (like MLP), while attention handles token mixing. Forcing them into the same routing decision means the router must compromise. In v4, PEER runs unconditionally for every token (replacing the standard MLP), while the router only decides the attention pathway (conv vs. attention vs. sink). Every token gets both PEER's feature transformation *and* its routed attention pathway, without competition.

The practical implication is refined: HRS implementations should not scale expert *count* (v3 shows this doesn't help), but should ensure that functionally distinct components (feature transformation vs. token mixing) occupy distinct architectural roles rather than competing through routing. The v3->v4 progression — same parameters, better architecture — is a cleaner demonstration of the HRS thesis: structure matters more than scale.

The v4 routing dynamics confirm the separation works. The 3-tier router stabilizes at conv 30% / attention 43% / sink 26%. Without PEER competing for routing weight, attention's share increases from 31% (v3) to 43% (v4) — the router is free to allocate attention where it helps most, rather than trading it off against expert computation.

Finding 9: Unconditional PEER + reduced routing achieves the best result

The v4 experiment (config 14) achieves the best perplexity of any configuration: 8.3 at end of Phase 4, a 65.2% reduction from the dense baseline. This result comes from a single architectural change relative to v3: moving PEER from a routed tier to an unconditional FFN and reducing the router from 4 tiers to 3.

The v4 training trajectory shows steady improvement through all four productive phases:

Phase	Steps	Best Val PPL	Tier Distribution (c/a/s)	Observation
P1 (Foundation)	1-8K	20.9	—	Backbone + conv at full LR, PEER at 0.01x

Phase	Steps	Best Val PPL	Tier Distribution (c/a/s)	Observation
P2 (Geometry)	8-16K	15.3	—	Router + locality at full LR
P3 (Specialization)	16-26K	10.4	—	Router + sink at full LR, PEER at 1.0x
P4 (Compression)	26-38K	**8.3**	30 / 43 / 26	Engrams activate — PPL plateaus at 8.3 from 31K–36K
P5 (Refinement)	38-50K	9.4	30 / 44 / 26	Regression: +1.3 PPL (moderate, less than v3's +4.87)

The 8.3 plateau during mid-to-late P4 (steps 31K, 34K, 36K all achieve 8.3) suggests the model reaches a stable optimum once engrams have fully activated. This contrasts with v1 and v3, where the best PPL occurs at a single point — v4's flatter optimum may indicate that separating PEER from routing produces a more stable loss landscape.

The v4 result has a specific implication for HRS architecture design: **functionally distinct operations should not compete through routing.** Feature transformation (PEER/MLP) and token mixing (attention) serve complementary roles. Routing should decide *how* tokens interact with each other (the attention pathway), not *whether* tokens undergo feature transformation (which should be unconditional). This is analogous to standard transformer design — every token passes through both attention and MLP — but with routing controlling only the attention pathway while PEER provides a universally superior FFN.

Finding 10: Prepending engrams outperforms in-place replacement

The v5 experiment (config 15) tests an alternative engram delivery mechanism: instead of prepending engram vectors as extra KV entries (increasing sequence length from 512 to 528), v5 blends upsampled engrams into the original sequence in-place using a learnable soft gate, keeping sequence length constant at 512. The gate sigma(alpha.(loss-theta)) uses EMA-cached per-token loss to identify stale windows (high loss = out-of-distribution context) and replaces them with upsampled engrams via a soft blend.

Result: v5 achieves best val_ppl 10.4 at step 37K — comparable to v1's full_hrs (10.64) but 25% worse than v4's 8.3 with the prepend mechanism.

The v5 training trajectory reveals an instructive dynamic:

Phase	Steps	Best Val PPL	Gate Mean	Loss Cache	Observation
P1 (Foundation)	1-8K	98.6	0.94	4.80	Gate saturated (cache » theta), near-full replacement
P2 (Geometry)	8-16K	35.3	0.75	3.86	Gate declining as model improves
P3 (Specialization)	16-26K	**10.5**	0.49	2.77	Cache crosses below threshold; best pre-P4 PPL
P4 (Compression)	26-38K	**10.4**	0.44	2.74	Gate params unfreeze; transient regression (->12.1) then recovery

The most notable behavior is that **the best PPL before P4 (10.5 at step 26K) nearly equals the final best (10.4 at step 37K)**. Unfreezing the gate parameters at P4 entry caused a transient regression from 10.5 to 12.1, and the model spent the entire 12K steps of P4 recovering. The learned threshold moved only from 3.00 to 3.04 — essentially the initial value was already near-optimal for the loss distribution, and making it learnable added optimization difficulty without benefit.

The explanation for v5's underperformance relative to v4 is architectural: **prepending adds information; replacing trades information.** When engrams are prepended, every token in the original sequence can attend to both the compressed engram vectors *and* all other original tokens — the model gets strictly more information. When engrams replace tokens in-place, the model gains compressed representations of stale windows but loses the residual signal those tokens carried, even if that signal was weak. The v4 sequence extension (512 -> 528) costs only 3% more compute/memory but preserves all original context while adding compressed summaries. The v5 replacement mechanism imposes a zero-sum trade-off that cannot match this.

This result establishes a practical design principle for engram systems: **deliver compressed memory as additional context (prepend/append), not as a substitute for existing context.** Even stale or high-loss tokens carry residual information worth preserving.

Summary

The ablation validates the HRS thesis with an important caveat: **routing alone is insufficient; routing plus compression is transformative.** The best HRS configuration (v4_full) achieves a best-of-run perplexity of 8.3 (65.2% below the dense baseline) at the end of Phase 4, with the v1 engram variants achieving 9.19–10.64 best-of-run (55.4–61.4% below baseline). The engram system is the decisive component, not the router — but the router creates the structured representations that make effective compression possible.

Five experimental axes strengthen this conclusion:

1. **Routing is necessary** (v2): Replacing learned routing with a fixed structural prior eliminates the benefit. Engrams without routing geometry worsen perplexity (+14.1% vs baseline), even with 262K sparse PEER experts.
2. **Expert capacity is not the bottleneck** (v3): Replacing v1's 4-expert MoE with PEER's 262K experts inside the routing framework yields the same perplexity (9.46 vs 9.19) with 3.5x more parameters. The routing-to-compression pipeline, not expert count, drives performance.
3. **Expert *role* matters** (v4): Moving PEER from a routed tier (competing with attention) to an unconditional FFN (complementing attention) improves perplexity from 9.46 to 8.3 with identical parameters — the best result across all experiments. Functionally distinct operations should not compete through routing.
4. **Phase 5 regression scales with coupling, not size** (v3 vs v4): v3 regresses +4.87 while v4 regresses only +1.30, despite identical parameter counts. The difference is that v4's PEER is decoupled from routing, confirming that disruption from encoder freezing is proportional to the degree of component coupling.
5. **Engram delivery mechanism matters** (v5): Replacing stale context in-place with upsampled engrams (v5: 10.4) underperforms prepending engrams as extra KV entries (v4: 8.3) by 25%. Prepending adds information; replacing trades information. Even stale tokens carry residual signal worth preserving.

Phase 5 (engram refinement) consistently degrades performance and should be considered optional or experimental at this model scale. The best practical configuration is a 4-phase protocol (P1–P4) with unconditional PEER and 3-tier routing.

---

14. Expected Benefits

Based on the experimental results above, we can now distinguish validated from projected benefits:

Validated (Section 13)

- representational diversity preserved through routing + compression (effective rank 43 vs 12.3 routing-only)
- 59.7–65.2% perplexity reduction over dense baseline at equivalent training budget (end-of-training 9.6–10.51 vs 23.83; best-of-run 8.3 vs 23.83)
- all compute tiers utilized without collapse (3-tier and 4-tier variants)
- phased training successfully sequences component activation (Phases 1–4)
- Phase 5 (engram refinement) degrades performance and should be considered optional
- learned routing is necessary — fixed structural priors cannot substitute (v2: engrams without routing worsen perplexity by 14.1%)
- expert capacity is not the bottleneck — 262K sparse experts yield the same perplexity as 4 small experts within the routing framework (v3: 9.46 vs 9.19 with 3.5x parameters)
- expert role separation improves performance — unconditional PEER + 3-tier routing outperforms PEER-as-routed-tier by 12.2% at identical parameter count (v4: 8.3 vs v3: 9.46)
- engram delivery as additional context (prepend) outperforms in-place replacement — v5's learnable context replacement (10.4) is 25% worse than v4's prepend mechanism (8.3), establishing that compressed memory should augment rather than substitute for existing context

Projected (require larger-scale validation)

- reduced cross-thread interference at longer context lengths
- better long-context reasoning on downstream tasks
- mostly O(n) compute at scale (routing overhead amortized)
- performance improvement via routing and compression rather than parameter growth

---

15. Minimal Model Hypothesis

We hypothesize:

A well-routed 1B HRS model can match or exceed dense 7B transformers **at equivalent FLOPs** on reasoning-heavy tasks.

This is a claim about compute efficiency, not parameter equivalence. Routing allows a smaller model to concentrate its capacity where it matters rather than spreading it uniformly. The claim is falsifiable: if a 1B HRS model cannot outperform a 1B dense model at equivalent FLOPs, routing overhead exceeds routing benefit and the architecture fails its own test.

The WikiText-103 ablation (Section 13) provides preliminary evidence: the full HRS system achieves best-of-run val_ppl 8.3 (v4, 175.6M params) at end of Phase 4 versus 23.83 for the dense baseline (38M parameters) — a 65.2% improvement. The v1 system achieves 9.19 with only 50M parameters (32% more than baseline), demonstrating a 61.4% improvement. However, we note that the router-only configurations (configs 3-5) show the opposite pattern: more parameters, worse perplexity. The hypothesis depends on the engram system scaling to larger models, which remains to be validated.

The v3->v4 progression provides the most instructive evidence. v3 (175.6M parameters) scales the expert tier by 130x within the same routing framework and achieves 9.46 — essentially the same as v1's 9.19. But v4 (175.6M parameters, identical count) restructures the architecture so PEER serves as an unconditional FFN rather than a routed tier, achieving 8.3 — a 12.2% improvement over v3 with zero additional parameters. This demonstrates precisely the hypothesis's claim: **better structure, not more parameters, drives improvement.** The v3->v4 comparison is the clearest example: same parameters, same PEER, same routing framework, same engrams — only the architectural role assignment changes, and perplexity drops by 12%.

The v5 experiment further reinforces this: replacing the engram delivery mechanism from prepend to in-place blend (v5: 10.4 vs v4: 8.3, both ~176M params) shows that how compressed memory is integrated matters as much as the compression itself. Augmenting context (prepend) strictly dominates substituting context (replace), even when the substituted tokens have high loss.

This would demonstrate that **structured compute allocation plus learned compression can substitute for scale**.

---

16. Ablation Order and Results

The ablation was designed to test each component's independent contribution. Results (Section 13) reveal a different pattern than predicted:

Step	Configuration	Predicted	Actual (Val PPL)	Outcome
1	Dense baseline	—	23.83	Reference
2	+ Dual-head (locality)	Measurable gain	24.08 (+1.0%)	Neutral — geometry shaping alone adds overhead without benefit at this scale
3	+ Router + tiered compute	Measurable gain	31.63 (+32.7%)	**Wrong** — router tax exceeds routing benefit without engrams
4	+ Sink channel	Measurable gain	31.90 (+33.9%)	Neutral — sink adds no measurable benefit at 512-token sequences
5	+ Phased training (core only)	Additional gain	37.84 (+58.8%)	**Wrong** — phased training without engrams hurts; phase transitions destabilize
6	+ Engrams (Phases 1-5)	Additional gain	12.04 (-49.5%)	**Dramatic gain** — engrams transform the architecture
7	+ Engram refinement (P5)	Additional gain	10.51 (-55.9%)	Gain vs baseline, but P5 itself regresses from best P4 PPL of 9.19

Step	Configuration	Predicted	Actual (Val PPL)	Outcome
8	+ TRC (window=8)	—	10.05 (-57.8%)	Modest smoothing benefit; smallest P5 regression (+0.62)

v2 counter-experiment: remove routing, keep structure.

Step	Configuration	Predicted	Actual (Val PPL)	Outcome
9	Fixed attn->conv backbone	Near baseline	25.7 (+7.8%)	Structural prior slightly worse than baseline
10	+ Dual-head	Measurable gain	26.2 (+9.9%)	**Wrong** — dual-head hurts without routing (opposite of v1)
11	+ PEER (262K experts)	Measurable gain	24.1 (+1.1%)	PEER nearly recovers baseline with sparse retrieval
12	+ Engrams + phased training	Additional gain	27.2 (+14.1%)	**Wrong** — engrams without routing geometry make things worse

v3 counter-experiment: scale expert capacity within routing.

Step	Configuration	Predicted	Actual (Best PPL)	Outcome
13	v1 routing + PEER (262K) + engrams	Beat 9.19	9.46 (-60.3%)	**Neutral** — 3.5x params, same perplexity; expert capacity not the bottleneck

v4 counter-experiment: separate PEER from routing.

Step	Configuration	Predicted	Actual (Best PPL)	Outcome
14	Unconditional PEER FFN + 3-tier routing + engrams	Beat 9.19	**8.3** (-65.2%)	**Best result** — separating FFN from routing lets both serve their natural roles

v5 counter-experiment: replace context instead of prepending.

Step	Configuration	Predicted	Actual (Best PPL)	Outcome
15	v4 + in-place engram replacement (learnable gate)	Beat 8.3	10.4 (-56.3%)	**Worse** — replacing tokens loses residual signal; prepending strictly adds information

Our prediction that "the core architecture should already demonstrate the thesis before extensions are added" was wrong. The core architecture (steps 2-5) consistently underperforms the dense baseline. The thesis is validated only when engrams are included. This has an important implication: **routing is necessary infrastructure for compression, not an independent source of improvement.** The value chain is routing -> structured geometry -> effective compression -> better perplexity, and removing the final link (compression) leaves only the cost of the preceding links.

The v2, v3, v4, and v5 experiments refine this further. v2 demonstrates that the first link (routing) cannot be replaced with a fixed structural prior — the geometry must be *learned* and *token-dependent*. v3 demonstrates that expanding the expert tier within an existing routing framework does not

improve the downstream compression — the bottleneck is routing quality and engram capacity, not expert count. v4 demonstrates that *how* components are composed matters as much as *what* components exist — moving PEER from routing competition to an unconditional role improves perplexity by 12.2% at identical parameter count. v5 demonstrates that *how* engrams are delivered matters — replacing context in-place (10.4) underperforms prepending as extra KV entries (8.3), establishing that compressed memory should augment context, not substitute for it.

The best practical protocol is a 4-phase training schedule (P1–P4) with unconditional PEER and 3-tier routing, achieving best-of-run val_ppl 8.3 (65.2% below baseline). Phase 5 consistently degrades performance across all engram variants and should be considered optional pending further investigation at larger scale.

Future work should test: (a) whether the router tax decreases at larger model scales where routing can save proportionally more compute; (b) whether TRC provides greater benefit at longer sequence lengths where temporal coherence spans are larger; (c) whether Phase 5 benefits from gentler encoder freezing (gradual LR decay rather than hard freeze) or longer training; (d) multi-seed runs to bound the substantial cross-run variance observed in engram configurations; (e) scaling engram capacity (more engrams per window, deeper encoder) as an alternative to scaling expert capacity; (f) whether the v4 pattern (unconditional FFN + reduced routing) extends to other expert types beyond PEER; (g) whether the v5 replacement mechanism benefits from longer sequences where the attention cost of prepended engrams becomes more significant (at 4K+ tokens, the prepend-vs-replace trade-off may shift).

---

17. Contributions

Core Architecture

1. Dual-head supervision for geometry shaping, with empirical evidence that single-objective training collapses 98.7% of representational capacity
2. Learned routing with softmax + Gumbel-softmax conservation constraints, balance loss, and entropy regularization — replacing Sinkhorn-Knopp, which we empirically demonstrated fails on non-square routing matrices
3. Convolution-first tiered operators with causal padding, supported by evidence that deeper attention heads compute near-static functions (92.6% variance explained by MLP approximations)
4. Sink channel for interference reduction, informed by measured salience dimensions (89% vs 34% recall by information density)
5. KV memory hierarchy treating the context window as a managed computer, not a passive buffer
6. Learnable tier output gate for magnitude-controlled residual integration, preventing effective rank collapse from tier output mismatch

Extensions

7. Temporal Routing Cache for amortized routing across coherent spans — empirically tested with causal moving average (window=8), showing modest stability benefit and the smallest Phase 5 regression among engram variants

8. Recursive thread engrams grounded in Hopfield energy minimization, with demonstrated 96% accuracy at 256x compression versus 80% for full-text RAG — and now validated as the decisive HRS component (up to 65.2% perplexity reduction vs dense baseline at best-of-run)
9. Convergence-triggered engram extraction via Jacobian sensitivity, replacing fixed schedules with learned signals
10. Iterative engram refinement exploiting the 24-28% untapped representational capacity in frozen representations — empirically demonstrated to regress at small model scale across all variants tested (see Section 13, Finding 3), establishing that Phase 5 should be considered optional

Training

11. Phased training protocol with differential learning rates and self-adaptive loss weighting, addressing the empirically demonstrated stacking failure mode
12. Empirical demonstration that routing without compression increases perplexity (the "router tax"), establishing that routing is infrastructure for compression rather than an independent source of improvement

Empirical Findings

13. Demonstration that dual-objective routing is a Nash equilibrium, not optimal transport — Sinkhorn-Knopp solves the wrong formalism for multi-head architectures, an important negative result for MoE routing research
14. Fifteen-configuration ablation across five experimental axes (v1 ablation, v2 structural prior, v3 expert scaling, v4 role separation, v5 engram delivery) on WikiText-103 demonstrating up to 65.2% perplexity reduction (23.83 -> 8.3 best-of-run) with the full HRS system
15. Phase dynamics analysis showing conv-dominated bootstrap (68% P1), engram-driven improvement (25% PPL drop in P4), and consistent P5 regression across all five engram variants (v1, v3, and v4) — establishing that the optimal practical protocol is 4 phases (P1–P4)
16. Demonstration that learned routing is necessary for engram effectiveness — fixed structural priors (v2) produce representations that engrams cannot exploit, worsening perplexity by 14.1% when engrams are added
17. Demonstration that expert capacity is not the bottleneck — 262K sparse PEER experts in the routing framework produce the same perplexity as 4 small MoE experts (9.46 vs 9.19) with 3.5x parameters, establishing routing geometry and engram compression as the performance-critical components
18. Demonstration that component role assignment matters — moving PEER from a routed tier to an unconditional FFN improves perplexity by 12.2% (9.46 -> 8.3) at identical parameter count, establishing that functionally distinct operations (feature transformation vs. token mixing) should not compete through routing
19. Demonstration that engram delivery mechanism matters — in-place replacement of stale context with upsampled engrams (v5: 10.4) underperforms prepending engrams as additional KV entries (v4: 8.3) by 25%, establishing that compressed memory should augment context rather than substitute for it

---

References (Author's Prior Work)

- Bee, M. (2025). "Attention Builds Maps, Then Reads Them: The Two Stages of Transformer Intelligence."
- Bee, M. (2025). "Binary Node Oscillation in Neural Networks: A Framework for Distinguishing Learning Failure from Active Computation."
- Bee, M. (2025). "Iterative Refinement: Breaking Through Convergence Plateaus in Neural Language Models."
- Bee, M. (2025). "Memory, Salience, and Why Your LLM Forgets: An Investigation We Actually Need to Run."
- Bee, M. (2025). "Self-Adaptive Loss Weighting Through Network Node Designation."
- Bee, M. (2025). "Static Functions Can Approximate Deep Attention Layers: Evidence from Per-Head Analysis."
- Bee, M. (2025). "The Context Window Is a Computer."
- Bee, M. (2025). "The Protocol Harness: An Operating System Written in Protocols."
- Bee, M. (2026). "Differentiable Time: When Neural Networks Learn They Are Finished."
- Bee, M. (2026). "Engrams: Learned Semantic Compression for Transformers."
- Bee, M. (2026). "Engrams Don't Inject Information — They Retrieve It."
- Bee, M. (2026). "Generative Auxiliary Training as Anti-Collapse Regularization in Large Language Models."
- Bee, M. (2026). "Golden Ratio Engram: A Corrected Account."
- Bee, M. (2026). "MCR2 as Language Model Regularizer: What Works, What Doesn't, and Why the Theory Oversells It."
- Bee, M. (2026). "Phased Specialization: Unlocking Hybrid Sequence Models via Optimization-Aware Training."
- Bee, M. (2026). "Recursive Refinement as Approximate Hopfield Dynamics."
- Bee, M. (2026). "Symmetry Constraints and Scaling Stability: A Noether-Inspired Framework for Neural Architecture Design."
- Bee, M. (2026). "The Four-Lane Highway: Making Sense of DeepSeek's mHC Paper."
- Bee, M. (2026). "We Stacked 3 AI Upgrades. The Combined System Was Worse Than Using Just 1."
- Bee, M. (2026). "What We Got Wrong About Engram Steering (And What We Actually Found)."